

گزارش کار تمرین کامپیوتری پنجم درس سیگنال و سیستم ها

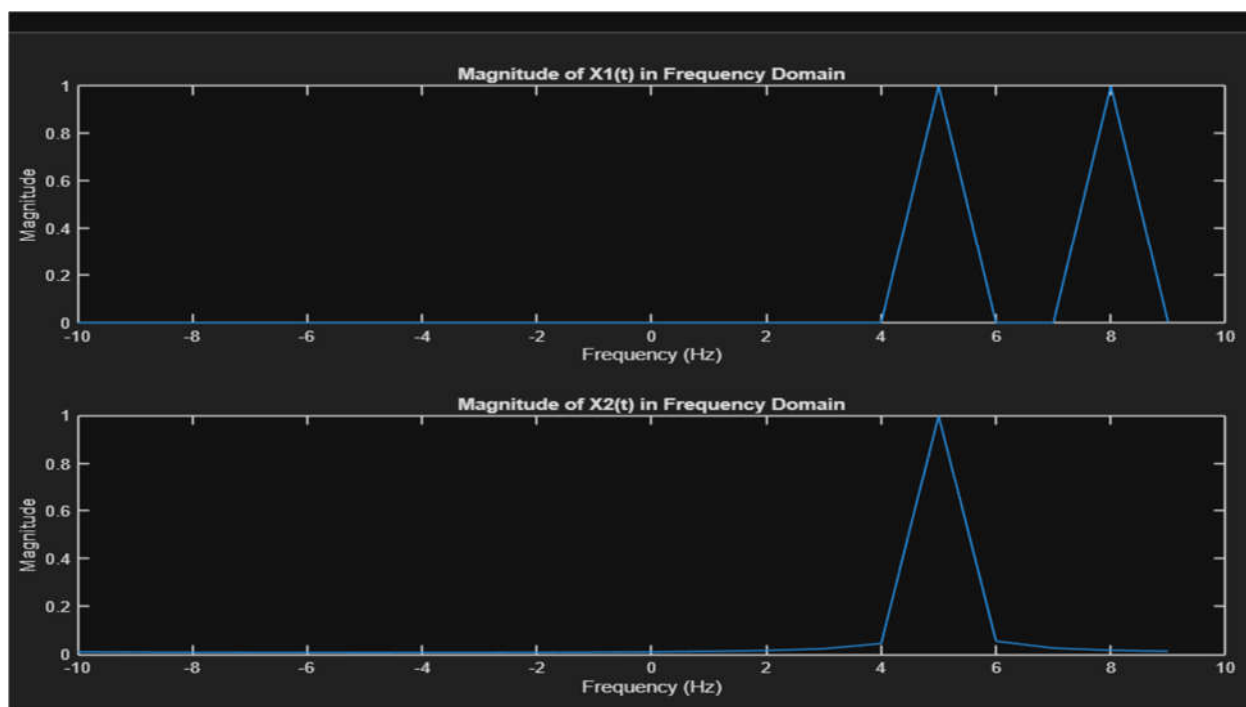
امیرحسین صباحی

810101556

بخش اول

(1.0)

کد frequency.m برای تحلیل دو سیگنال گسسته با فرکانس‌های مختلف در حوزه فرکانس استفاده می‌شود تا تأثیر رزولوشن فرکانسی را بر توانایی تفکیک سیگنال‌ها در تبدیل فوریه بررسی کند. دو سیگنال به صورت ترکیبی از دو فرکانس مختلف تعریف شده‌اند: سیگنال اول شامل فرکانس‌های ۵ و ۸ هرتز، و سیگنال دوم شامل فرکانس‌های ۵ و ۵.۱ هرتز است. با استفاده از تبدیل فوریه گسسته (fft) و جابجایی فرکانس‌ها با دستور fftshift، طیف فرکانسی هر سیگنال محاسبه و نرمال‌سازی می‌شود. نتایج نشان می‌دهد که در سیگنال اول، دو قله در فرکانس‌های ۵ و ۸ هرتز ظاهر می‌شود، در حالی که در سیگنال دوم، به دلیل تفاوت کمتر از رزولوشن فرکانسی $\delta f = 1$ هرتز، تنها یک قله در ۵ هرتز مشاهده می‌شود.



```

F:\matlab\frequeuncy.m
1      T = 1;
2      fs = 20;
3      t = 0:1/fs:T-1/fs;
4      N = length(t);
5      delta_f = 1/T;
6
7      x1 = exp(1j*2*pi*5*t) + exp(1j*2*pi*8*t);
8      x2 = exp(1j*2*pi*5*t) + exp(1j*2*pi*5.1*t);
9
10     X1 = fftshift(fft(x1));
11     X2 = fftshift(fft(x2));
12
13     f = (-fs/2):(fs/N):(fs/2)-(fs/N);
14
15     X1_abs = abs(X1)/max(abs(X1));
16     X2_abs = abs(X2)/max(abs(X2));
17
18     figure;
19     subplot(2,1,1);
20     plot(f, X1_abs);
21     title('Magnitue of X1(t) in Frequency Domain');
22     xlabel('Frequency (Hz)');
23     ylabel('Magnitue');
24
25     subplot(2,1,2);
26     plot(f, X2_abs);
27     title('Magnitue of X2(t) in Frequency Domain');
28     xlabel('Frequency (Hz)');
29     ylabel('Magnitue');
30

```

1.1

Signal.m

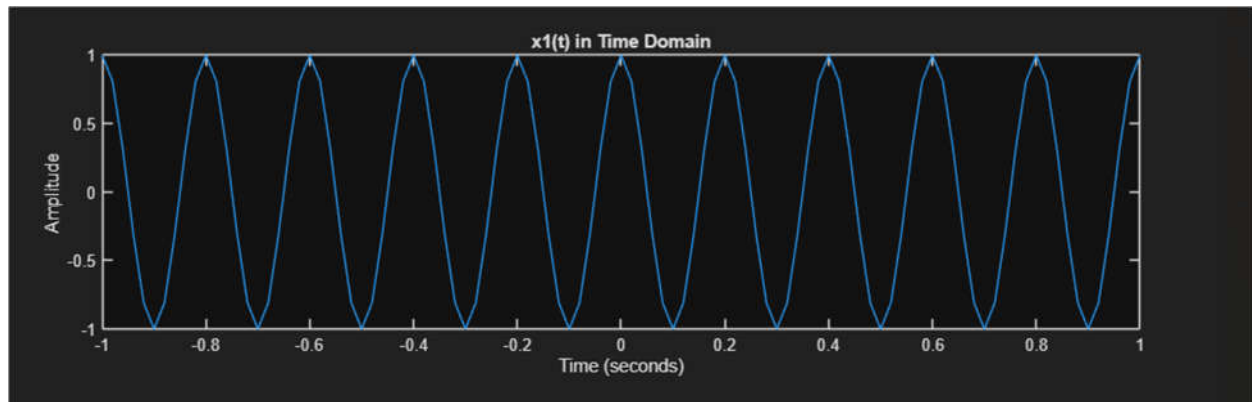
این کد برای تحلیل سیگنال $x_1(t) = \cos(10\pi t)$ در دو حوزه زمان و فرکانس استفاده می‌شود. ابتدا سیگنال در بازه زمانی از $t_{start} = -1$ تا $t_{end} = 1$ ثانیه با فرکانس نمونه‌برداری $f_s = 50$ هرتز تعریف شده و در حوزه زمان رسم می‌شود. سپس، تبدیل فوریه گسسته سیگنال با استفاده از دستور `fft` محاسبه و با دستور `fftshift` جابجا می‌شود. اندازه تبدیل فوریه نرمال‌سازی می‌شود تا ماکزیمم آن برابر با ۱ باشد و سپس در حوزه فرکانس رسم می‌شود.

```

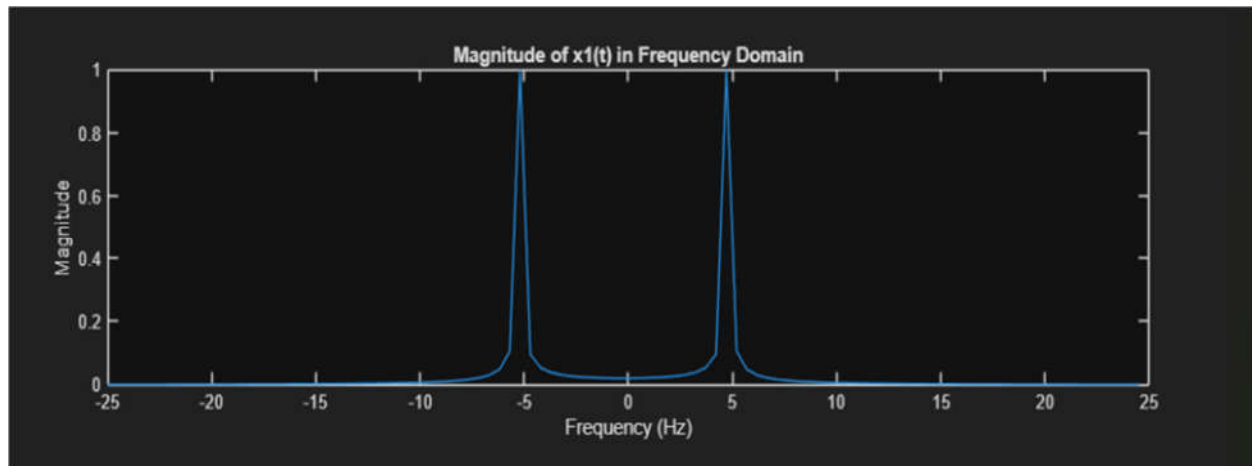
F:\matlab\signal.m
1      fs = 50;
2      t_start = -1;
3      t_end = 1;
4      t = t_start:1/fs:t_end;
5      x1 = cos(10*pi*t);
6
7      figure;
8      subplot(2,1,1);
9      plot(t, x1);
10     title('x1(t) in Time Domain');
11     xlabel('Time (seconds)');
12     ylabel('Amplitude');
13     xlim([t_start t_end]);
14
15     X1 = fftshift(fft(x1));
16     N = length(t);
17     f = (-fs/2):(fs/N):(fs/2)-(fs/N);
18
19     X1_abs = abs(X1)/max(abs(X1));
20
21     subplot(2,1,2);
22     plot(f, X1_abs);
23     title('Magnitude of x1(t) in Frequency Domain');
24     xlabel('Frequency (Hz)');
25     ylabel('Magnitude');
26     xlim([-fs/2 fs/2]);
27

```

(الف)



(ب)



تبدیل فوریه سیگنال $x_1(t) = \cos(10\pi t)$ یک قله در فرکانس 5 هرتز و یک قله آینه‌ای در فرکانس -5 هرتز نشان می‌دهد. این نتیجه با ویژگی‌های سیگنال‌های تک‌تُن تطابق دارد، زیرا سیگنال $x_1(t)$ تنها فرکانس 5 هرتز را شامل می‌شود و مطابق با تبدیل فوریه گسسته، یک قله در فرکانس مثبت و یک قله در فرکانس منفی مشاهده می‌شود. نرمال‌سازی اندازه تبدیل فوریه نیز باعث می‌شود که قله‌ها به مقدار 1 نرمال شوند، که در نمودار مشاهده می‌شود.

1.2

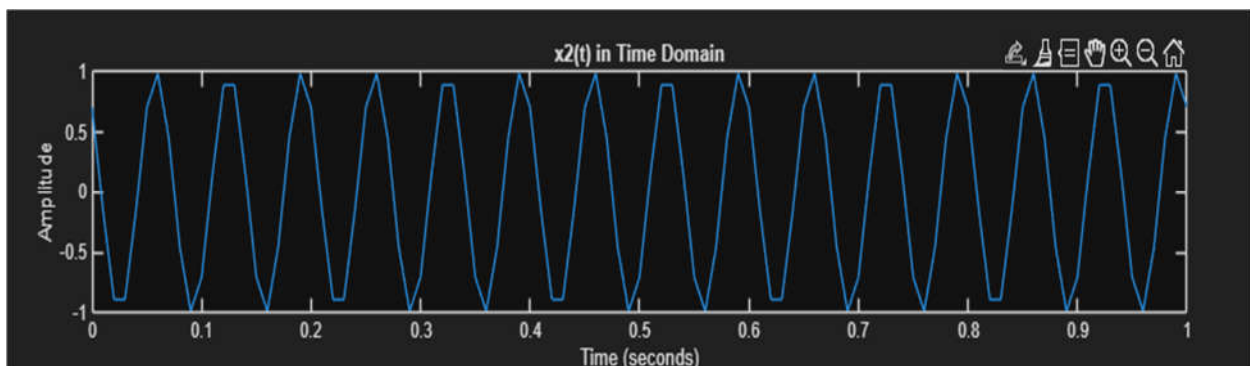
در ادامه کد `Signal.m` برای تحلیل سیگنال $x_2(t) = \cos(30\pi t + \pi/4)$ در سه مرحله طراحی شده است. ابتدا، سیگنال در بازه زمانی 0 تا 1 ثانیه با فرکانس نمونه‌برداری 100 هرتز تعریف و در حوزه زمان رسم می‌شود. سپس، تبدیل فوریه گسسته سیگنال با استفاده از دستور `fft` محاسبه و با `fftshift` جابجا می‌شود. اندازه تبدیل فوریه نرمال‌سازی می‌شود تا ماکزیمم آن برابر با 1 باشد و در حوزه فرکانس رسم می‌شود. در نهایت، فاز سیگنال محاسبه شده و فاز نقاطی که اندازه آن‌ها کمتر از یک آستانه ($1e-6$) است، صفر می‌شود. سپس فاز سیگنال به صورت ضربی از π نمایش داده می‌شود. این کد به‌طور کامل سیگنال را از لحاظ زمانی، فرکانسی و فازی تجزیه و تحلیل می‌کند.

```

F:\matlab\signal.m
28
29     fs = 100;
30     t_start = 0;
31     t_end = 1;
32     t = t_start:1/fs:t_end;
33     x2 = cos(30*pi*t + pi/4);
34
35     figure;
36     subplot(3,1,1);
37     plot(t, x2);
38     title('x2(t) in Time Domain');
39     xlabel('Time (seconds)');
40     ylabel('Amplitude');
41     xlim([t_start t_end]);
42
43     X2 = fftshift(fft(x2));
44     N = length(t);
45     f = (-fs/2):(fs/N):(fs/2)-(fs/N);
46
47     X2_abs = abs(X2)/max(abs(X2));
48
49     subplot(3,1,2);
50     plot(f, X2_abs);
51     title('Magnitude of x2(t) in Frequency Domain');
52     xlabel('Frequency (Hz)');
53     ylabel('Magnitude');
54     xlim([-fs/2 fs/2]);
55
56     tol = 1e-6;
57     X2(abs(X2) < tol) = 0;
58     theta = angle(X2);
59
60     subplot(3,1,3);
61     plot(f, theta/pi);
62     title('Phase of x2(t) in Frequency Domain');
63     xlabel('Frequency (Hz)');
64     ylabel('Phase / \pi');
65     xlim([-fs/2 fs/2]);

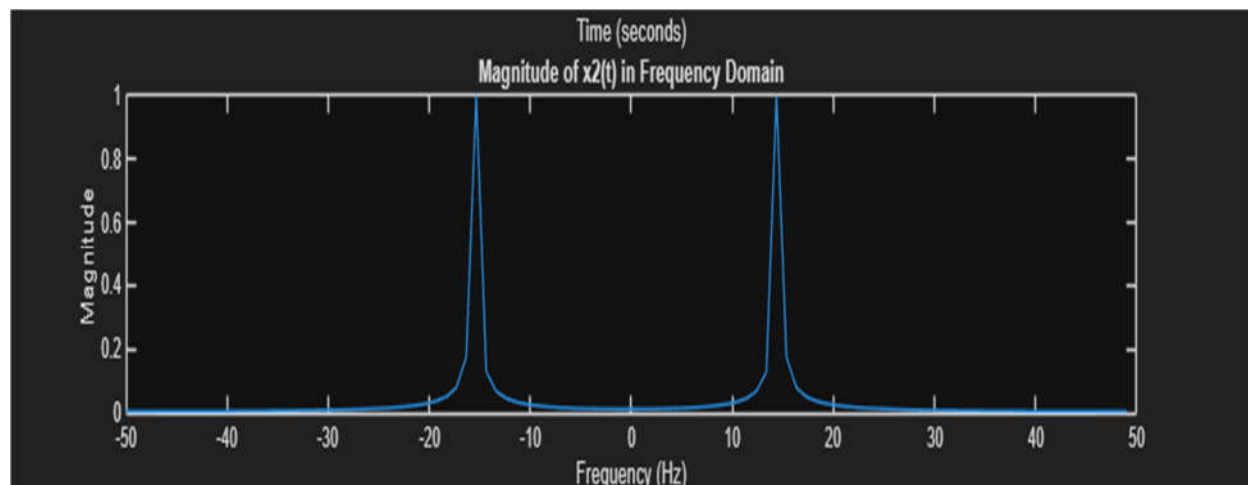
```

(الف)



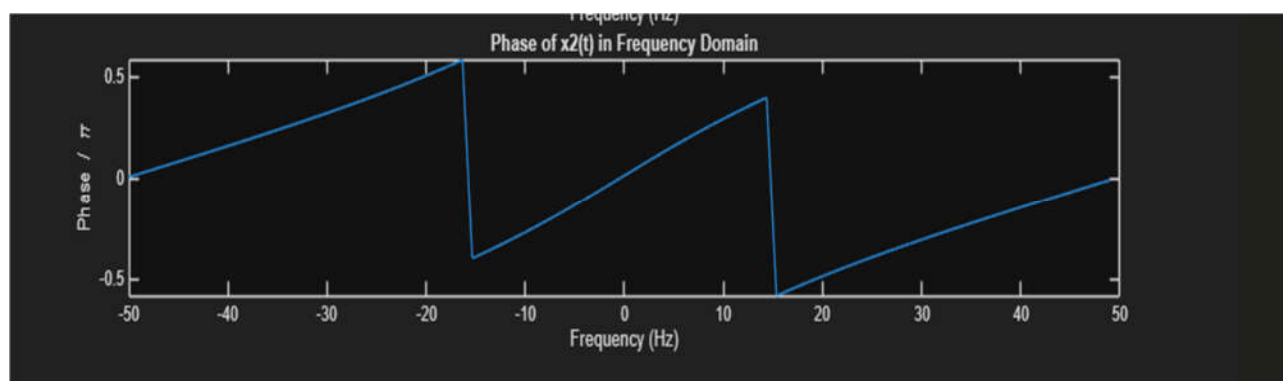
(ب)

یک قله در فرکانس 15 هرتز و یک قله آینه‌ای در فرکانس -15 هرتز دارد. این تطابق کامل با دانسته‌های قبلی است.



(ج)

فاز تبدیل فوریه نشان‌دهنده‌ی تغییرات فاز سیگنال در حوزه فرکانس است. به دلیل وجود فاز اولیه $p/4$ در سیگنال، فاز در فرکانس‌های اصلی تغییرات مشخصی دارد و در فرکانس‌هایی که اندازه تبدیل فوریه ناچیز است، فاز صفر می‌شود.



بخش دوم

(2.1)

make_mapset.m

```
F:\matlab\make_mapset.m
1 function Mapset = make_mapset()
2 symbols = [arrayfun(@char, 'a':'z', 'UniformOutput', false), {' '}, {'.'}, {','}, {'!'}, {';'}, {'"'}, {'\'}];
3 Mapset = cell(2,32);
4 for k = 1:32
5     Mapset{1,k} = symbols{k};
6     Mapset{2,k} = dec2bin(k-1,5) - '0';
7 end
8 end
9
```

(2.2)

این کد تابع **coding_freq** را برای کدگذاری پیام به سیگنال‌های سینوسی با سرعت مشخص شده پیاده‌سازی می‌کند. ابتدا پیام ورودی به رشته باینری تبدیل می‌شود و سپس برای هر بخش از پیام که بر اساس سرعت ارسال اطلاعات (bitrate) تقسیم شده است، یک سیگنال سینوسی با فرکانس متفاوت تولید می‌شود. این فرکانس‌ها با استفاده از تبدیل باینری به عدد معادل تعیین می‌شوند. در نهایت، سیگنال‌های کدگذاری شده به یکدیگر متصل می‌شوند تا سیگنال نهایی که پیام کدگذاری شده را نمایش می‌دهد، تشکیل شود.

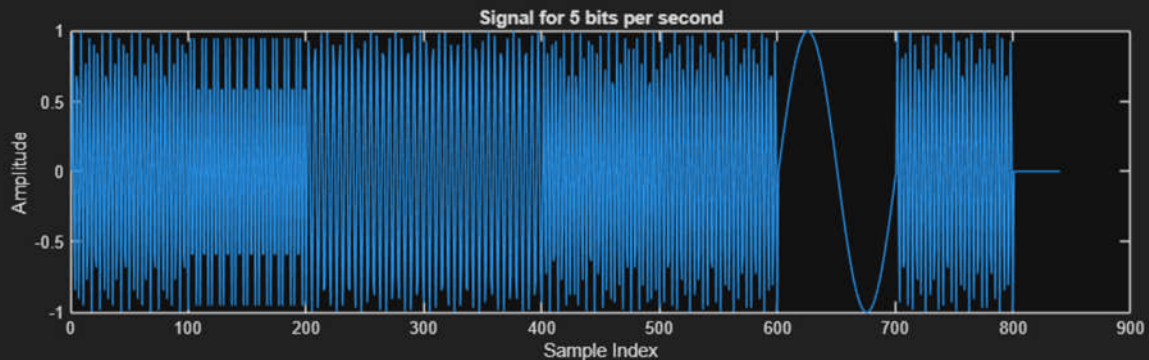
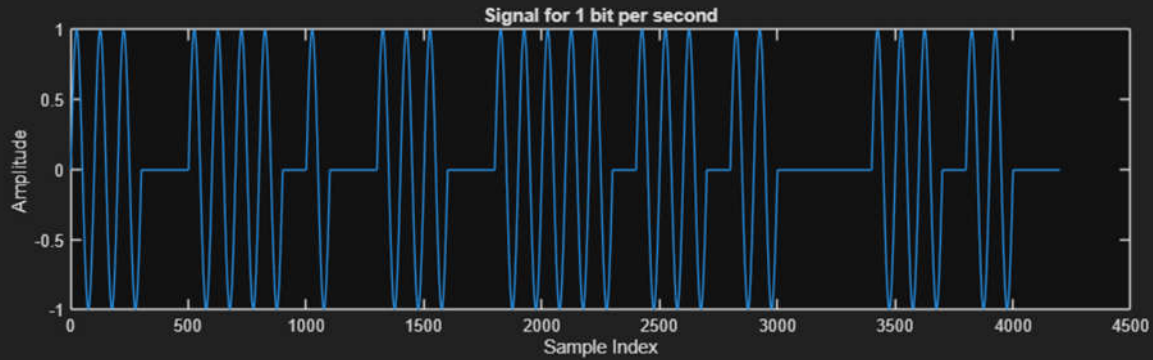
```
1 function sig = coding_freq(msg, bitrate)
2     msgBin = '';
3     for i = 1:length(msg)
4         msgBin = [msgBin, dec2bin(msg(i), 5)];
5     end
6     R = bitrate;
7     fs = 100;
8     t = 0:1/fs:1-1/fs;
9     numSymbols = length(msgBin) / R;
10
11     base = 2 * sin(2 * pi * t);
12
13     sig = zeros(1, numSymbols * length(t));
14
15     for k = 1:numSymbols
16         idx = (k-1)*R + 1 : k*R;
17         bits = msgBin(idx);
18         freq = bin2dec(bits);
19         sig((k-1)*length(t) + 1:k*length(t)) = sin(2 * pi * freq * t);
20     end
21 end
22
```


F:\matlab\run.m

```

1      msg = 'signal';
2      bitrate1 = 1;
3      bitrate5 = 5;
4
5      sig1 = coding_freq(msg, bitrate1);
6      sig5 = coding_freq(msg, bitrate5);
7
8      figure;
9      subplot(2,1,1);
10     plot(sig1);
11     title('Signal for 1 bit per second');
12     xlabel('Sample Index');
13     ylabel('Amplitude');
14
15     subplot(2,1,2);
16     plot(sig5);
17     title('Signal for 5 bits per second');
18     xlabel('Sample Index');
19     ylabel('Amplitude');
20

```



تابع `decoding_freq` ابتدا سیگنال ورودی را با فرض $fs=100$ به بلوک‌های یک‌ثانیه‌ای (هر بلوک 100 نمونه) تقسیم می‌کند و برای هر بلوک تبدیل فوریه را با `fft` و `fftshift` محاسبه می‌کند تا طیف حول فرکانس صفر مرتب شود. چون در کدگذاری فرکانسی برای هر سمبل فقط از $R/2$ فرکانس مشخص استفاده شده، تابع همان مجموعه فرکانس‌های مجاز را با `choose_freqs` می‌سازد و اندیس دقیق آن‌ها را روی محور فرکانسی f پیدا می‌کند. سپس در هر بلوک، فقط دامنه‌ی FFT روی همین اندیس‌های کاندید بررسی می‌شود و فرکانسی که بیشترین قله را دارد به عنوان فرکانس اصلی آن بلوک انتخاب می‌شود؛ شماره‌ی این فرکانس (`val`) همان مقدار دسیمال سمبل و معادل R بیت آن بخش از پیام است.

بعد از اینکه برای همه‌ی بلوک‌ها R بیت استخراج شد، کل رشته بیت به نزدیک‌ترین مضرب 5 بریده می‌شود تا بتوان آن را به گروه‌های 5 بیتی (هر کاراکتر) تبدیل کرد و `reshape` بدون خطا انجام شود. سپس یک ماتریس `codes` از `Mapset` ساخته می‌شود که در آن هر سطر کد 5 بیتی یک کاراکتر از مجموعه 32 تایی است. تابع برای هر گروه 5 بیتی استخراج‌شده، سطر متناظر را در `codes` پیدا می‌کند و کاراکتر متناظر را از `Mapset` برمی‌دارد و کنار هم می‌چیند تا رشته نهایی `msg` ساخته شود؛ بنابراین خروجی تابع همان پیام متنی اولیه است (اگر نویز یا خطای تصمیم‌گیری باعث انتخاب فرکانس اشتباه نشده باشد).

```

F:\matlab\decoding_freq.m
1 function msg = decoding_freq(sig, bitrate)
2     Mapset = make_mapset();
3     R = bitrate;
4     fs = 100;
5     N = fs;
6
7     numSym = floor(length(sig)/N);
8     sig = sig(1:numSym*N);
9
10    M = 2^R;
11    freqs = choose_freqs(M, fs);
12
13    f = (-fs/2):(fs/N):(fs/2 - fs/N);
14    candIdx = zeros(1, M);
15    for k = 1:M
16        candIdx(k) = find(f == freqs(k), 1);
17    end
18
19    bits = zeros(1, numSym*R);
20    for i = 1:numSym
21        s = sig((i-1)*N+1:i*N);
22        Y = fftshift(fft(s));
23        mag = abs(Y);
24
25        [~, midx] = max(mag(candIdx));
26        val = midx - 1;
27
28        b = dec2bin(val, R) - '0';
29        bits((i-1)*R+1:i*R) = b;
30    end
31
32    bits = bits(1:floor(length(bits)/5)*5);
33    bits5 = reshape(bits, 5, []).';
34
35    codes = zeros(32,5);
36    for k = 1:32
37        codes(k,:) = Mapset{2,k};
38    end
39

```

```

F:\matlab\decoding_freq.m
39
40    msgChars = repmat(' ', 1, size(bits5,1));
41    for j = 1:size(bits5,1)
42        idx = find(ismember(codes, bits5(j,:), 'rows'), 1);
43        msgChars(j) = Mapset{1,idx};
44    end
45
46    msg = msgChars;
47 end
48
49 function freqs = choose_freqs(M, fs)
50     fmax = fs/2 - 1;
51     freqs = round(linspace(2, fmax, M));
52     freqs = unique(freqs, 'stable');
53     if length(freqs) < M
54         pool = setdiff(1:fmax, freqs, 'stable');
55         freqs = [freqs, pool(1:(M-length(freqs)))];
56     else
57         freqs = freqs(1:M);
58     end
59 end
60

```

در run.m

```
msg0 = 'signal';  
s1 = coding_freq(msg0, 1);  
d1 = decoding_freq(s1, 1);  
s5 = coding_freq(msg0, 5);  
d5 = decoding_freq(s5, 5);  
fprintf('bitrate = 1 --> decoded = %s\n', d1);  
fprintf('bitrate = 5 --> decoded = %s\n', d5);
```

>> run

```
bitrate = 1 --> decoded = signal  
bitrate = 5 --> decoded = signal
```

(2.5

در run.m

```
%2.5  
msg0 = 'signal';  
s1 = coding_freq(msg0, 1);  
s5 = coding_freq(msg0, 5);  
sigma = sqrt(0.0001);  
r1 = s1 + sigma*randn(size(s1));  
r5 = s5 + sigma*randn(size(s5));  
d1 = decoding_freq(r1, 1);  
d5 = decoding_freq(r5, 5);  
fprintf('bitrate = 1 , decoded = %s\n', d1);  
fprintf('bitrate = 5 , decoded = %s\n', d5);
```

خروجی

```
bitrate = 1 , decoded = signal  
bitrate = 5 , decoded = signal
```

با نویز گوسی با میانگین صفر و واریانس 0.0001 (یعنی انحراف معیار 0.01)، پیام signal همچنان درست استخراج می‌شود، چون توان نویز نسبت به دامنه سیگنال کم است و قله فرکانس اصلی در FFT هر بازه یک‌ثانیه‌ای هنوز از بقیه فرکانس‌های کاندید بزرگ‌تر می‌ماند. بنابراین برای $\text{bitrate}=1$ خروجی decoding برابر signal شود و برای $\text{bitrate}=5$ هم signal درست بازیابی شود، هرچند احتمال خطا برای $\text{bitrate}=5$ نسبت به $\text{bitrate}=1$ بیشتر است چون تعداد فرکانس‌های کاندید بیشتر است و تصمیم‌گیری سخت‌تر می‌شود

2.6

با افزایش تدریجی واریانس نویز، ابتدا دقت رمزگشایی $\text{bitrate}=5$ شروع به افت می‌کند و زودتر از $\text{bitrate}=1$ به خطا می‌افتد؛ در حالی که $\text{bitrate}=1$ تا واریانس‌های بزرگ‌تری همچنان پیام signal را درست بازیابی می‌کند. بنابراین $\text{bitrate}=1$ مقاوم‌تر به نویز است. این نتیجه با مقدمه همخوانی دارد: وقتی bitrate بالاتر است، تعداد فرکانس‌های کاندید (2^R) بیشتر می‌شود و فاصله‌ی فرکانس‌های انتخابی معمولاً کمتر می‌شود، پس در حضور نویز احتمال اینکه قله‌ی FFT بین فرکانس‌های کاندید جابه‌جا شود و تصمیم‌گیری اشتباه شود بیشتر خواهد بود.

1.5

```
bitrate = 1 , decoded = signal
bitrate = 5 , decoded = signal
```

3.5

```
bitrate = 1 , decoded = signal
bitrate = 5 , decoded = si,nap
```

5

```
bitrate = 1 , decoded = sienal
bitrate = 5 , decoded = big.al
```

(2.7)

بیشترین واریانس قابل تحمل برای $\text{bitrate}=5$ و $\text{bitrate}=1$ مشاهده شد.

(2.9)

اگر نرخ نمونه‌برداری f_s را زیاد کنی ولی پهنای باند مصرفی را زیاد نکنی (یعنی همان بازه فرکانسی و همان فاصله فرکانس‌های انتخابی را نگه داری)، ذاتاً مقاومت به نویز بهتر نمی‌شود؛ چون عامل اصلی مقاوم شدن در این روش همان بیشتر شدن فاصله بین فرکانس‌های کاندید و در نتیجه کمتر شدن احتمال جابه‌جایی قله‌ها در تصمیم‌گیری است. افزایش f_s بدون افزایش پهنای باند فقط تعداد نمونه‌های بیشتری از همان سیگنال باند-محدود نویزی به تو می‌دهد و اگر روش تصمیم‌گیری را همان قبلی نگه داری، جداسازی فرکانس‌ها بهتر نمی‌شود.